JAVA SDE-2 INTERVIEW PREPARATION SERIES

DATA STRUCTURES AND ALGORITHMS - BOOK 13 OF 17 - STUDY STEP DSA-13

Trees, Binary Search Trees, and Tries

Traversal Contracts, Path State, Ordering, Balance, and Prefix Search

BY

Vinay Reddy Kalluri

Model recursive tree contracts, own traversal state correctly, exploit BST ordering, and use tries for prefix-shaped search.

DFS | BFS | BST | LCA | BALANCE | TRIES | SERIALIZATION

JAVA 21 | INTERVIEW CORE | SDE-2 FOLLOW-UPS | PRINTABLE EDITION

Series edition - Updated 2026-08-02

Open educational interview-preparation edition

Start Here - Your Route Through This Volume

> **60-second entry rule:** If two or more recognition signals below expose a gap, begin with Chapter 1. If the prerequisites are weak, step back to `DSA 12 - Binary Search`. If you can already explain every outcome without notes, jump to Part II and prove it through the practice ladder.

Tree interviews test recursion, state ownership, structural invariants, and the ability to translate ordering into pruning.

Readiness map

Decision	Evidence
START HERE	The input is hierarchical or path-shaped; Subtree results combine recursively; Ordering can eliminate one branch; Keys share prefixes.
PREPARE FIRST	JAVA-01 and DSA-01 through DSA-12; Recursion, queues, and binary search.
MOVE ON WHEN	Implement and explain recursive and iterative traversals; Reason about depth, paths, LCA, and BST invariants; Serialize, reconstruct, and validate tree structure; Build and query a trie.

Choose the route that fits today

1. **Learning:** read in order, type the representative Java examples, and complete each dry run.
2. **Revision or rehearsal:** start at the first decision map, invariant, or failure mode you cannot explain; if none expose a gap, go directly to Part II and prove readiness without notes.

DSA Segment Position

DSA 12 - Binary Search	**YOU ARE HERE** **DSA 13 - Trees, BSTs, and Tries**	DSA 14 - Heaps and Priority Queues

A note from Vinay

Tree code becomes clearer when every method states what its subtree returns. For balanced and range structures, trace the metadata update with the same care as the pointer update.

Contents

This contents page covers only the current focused PDF. Section-level navigation is available through PDF bookmarks.

CONTENTS NAVIGATION

Part I - Core Learning

SDE-2 CORE CHAPTER

Chapter 1 - Tree Foundations: Vocabulary, Traversal, and Recursive Contracts

A tree is a connected acyclic structure. In interview code, a binary-tree node usually owns references to at most two children.

JAVA

```java
static final class TreeNode {
    int value;
    TreeNode left;
    TreeNode right;

    TreeNode(int value) {
        this.value = value;
    }
}
```

Vocabulary on one picture

TEXT

```
            8                root
          /   \
         3     10            children of 8
        / \      \
       1   6      14          leaves include 1, 6, 14

path 8 -> 3 -> 6 has two edges
depth(6) = 2 when root depth is 0
height(3) = 1 when leaf height is 0
subtree rooted at 3 contains 3, 1, and 6
```

State whether height counts edges or nodes. Either convention can be correct, but formulas and empty-tree base cases differ.

The recursive subtree contract

For node-counting, define:

TEXT

```
count(node) returns the number of nodes in the subtree rooted at node
```

JAVA

```java
static int count(TreeNode node) {
    if (node == null) {
        return 0;
    }
    return 1 + count(node.left) + count(node.right);
}
```

The null reference represents an empty subtree. Each non-null call combines the answers of two strictly smaller subtrees.

For height measured in nodes:

JAVA

```java
static int height(TreeNode node) {
    return node == null ? 0 : 1 + Math.max(height(node.left), height(node.right));
}
```

The recursion depth is O(h), where h is tree height. It is O(log n) only for a balanced tree and O(n) for a chain.

Traversal orders answer different questions

For each node:

- preorder: process, left, right;
- inorder: left, process, right;
- postorder: left, right, process;
- level order: increasing distance from root using a queue.

TEXT

```
        2
       / \
      1   3

preorder:  2,1,3
inorder:   1,2,3
postorder: 1,3,2
level:     2,1,3
```

Postorder naturally fits information synthesized from children. Preorder fits carrying state from parent to child. Inorder reveals sorted order only for a valid binary search tree, not for every binary tree.

Iterative DFS and explicit state

An iterative preorder stack is direct: push root, pop, process, push right then left so left is processed first.

Iterative inorder needs a cursor plus a stack of ancestors:

JAVA

```java
static List<Integer> inorder(TreeNode root) {
    List<Integer> output = new ArrayList<>();
    Deque<TreeNode> stack = new ArrayDeque<>();
    TreeNode current = root;
    while (current != null || !stack.isEmpty()) {
        while (current != null) {
            stack.push(current);
            current = current.left;
        }
        current = stack.pop();
        output.add(current.value);
        current = current.right;
    }
    return output;
}
```

The stack contains ancestors whose left subtree is complete but whose node/right subtree still needs processing.

Breadth-first traversal

JAVA

```java
static List<List<Integer>> levels(TreeNode root) {
    if (root == null) {
        return List.of();
    }
    List<List<Integer>> result = new ArrayList<>();
    Deque<TreeNode> queue = new ArrayDeque<>();
    queue.addLast(root);
    while (!queue.isEmpty()) {
        int levelSize = queue.size();
        List<Integer> level = new ArrayList<>(levelSize);
        for (int i = 0; i < levelSize; i++) {
            TreeNode node = queue.removeFirst();
            level.add(node.value);
            if (node.left != null) queue.addLast(node.left);
            if (node.right != null) queue.addLast(node.right);
        }
        result.add(level);
    }
    return result;
}
```

Snapshot `levelSize` before processing. The queue grows with children for the next level.

WHY IT WORKS | Binary search tree invariant

A common distinct-key BST contract is:

```text
every key in left subtree < node key < every key in right subtree
```

This is a global subtree rule. Checking only immediate children is insufficient; a deep descendant can violate an ancestor bound. Duplicate policy must be explicit.

BST operations cost O(h), not automatically O(log n). Without balancing, sorted insertion can create a height-n chain.

Tries from zero

A trie stores keys by prefix. A node represents a consumed prefix; outgoing edges represent next symbols; a terminal flag distinguishes a complete word from a prefix.

```text
root
  c
  |
  a -- terminal for "ca" if allowed
 / \
t   r
*   *      words "cat" and "car"
```

Operation cost is O(L) in key length under a suitable child representation. Memory depends on total created prefix nodes and child-container overhead.

State-ownership clinic

- Local accumulators returned from children avoid stale instance fields.
- A path list shared down DFS must be restored on return.
- A global `previous` pointer for BST validation must be reset before reuse and is unsafe for concurrent calls.
- Serialization must encode null structure, not only values, unless another invariant makes shape recoverable.

TRY IT | Foundation checkpoint

1. Give a one-sentence contract for tree height.
2. Why is recursive space O(h) rather than always O(log n)?
3. What unresolved work does an iterative inorder stack contain?
4. Why is parent-child comparison insufficient for BST validation?
5. How does a trie distinguish a word from a prefix?

SDE-2 CORE CHAPTER

Chapter 2 - Trees, Binary Search Trees, and Tries for SDE-2

Tree questions combine recursive contracts with representation choices. A traversal is easy to memorize; the interview signal is whether you can say what a helper returns, where state lives, and why information from children is sufficient. Binary search trees add an ordering invariant that spans entire subtrees. Tries add prefix state and a potentially expensive branching representation. This chapter keeps heaps and ordered-map APIs out of scope so tree ownership and search invariants receive full attention.

Recognition and traversal map

Prompt signal	Traversal/state	Reason
parent before children, copy/serialize	preorder DFS	decision is made on entry
sorted values from a BST	inorder DFS	left, node, right follows ordering
children before parent, aggregate height	postorder DFS	parent depends on child summaries
levels, minimum unweighted depth	BFS	FIFO preserves distance layers
path sum / root-to-leaf condition	DFS with path state	state belongs to one current path
nearest shared ancestor	postorder returned evidence	combine whether targets occur below
ordered lookup/update	BST comparison	discard a full subtree per comparison
prefix lookup/autocomplete	trie walk by symbol	path itself is the prefix

Before choosing recursion, estimate height. A balanced tree with `n` nodes has height `O(log n)`; a skewed tree has height `n`. Java does not guarantee tail-call elimination. An iterative traversal or explicit frame stack is the safe production choice for adversarial depth.

The structural and ownership model

A conventional tree is acyclic, has one root, and gives every non-root node exactly one parent. A Java `Node` object does not enforce those properties. Callers can create cycles or share a child between parents, turning the structure into a graph. Most interview algorithms assume a proper tree; state it.

Mutation contracts matter. BST insert/delete below reuse and rewire input nodes. Reconstruction allocates new nodes. Traversals do not mutate. A node exposed to multiple owners cannot be safely rewired without coordination. Production APIs often hide nodes, use immutable nodes, or confine mutations behind a tree abstraction.

Family 1: depth-first and breadth-first traversals

Recursive traversals

For preorder, the contract can be: "append the preorder sequence of `node`'s subtree and leave earlier output unchanged." The base case for `null` appends nothing. Append the node, then recursively append left and right. Inorder moves the append between children; postorder moves it after children.

Correctness follows by structural induction. An empty tree is correct. Assuming each child call returns its correct traversal, placing the root in the traversal-defined position yields the correct sequence for the parent. Every call moves to a child, so an acyclic finite tree terminates.

Time is `O(n)` because every node is visited once. Call-stack space is `O(h)` for height `h`; returned output is `O(n)` and is not auxiliary workspace.

Iterative inorder

Use a stack and cursor. The invariant is that the stack holds a path of nodes whose left subtrees have been scheduled or processed but whose own values have not yet been emitted; `current` heads a not-yet-descended subtree. Push the entire left spine. Pop one node, emit it, then explore its right subtree.

For tree `2` with children `1` and `3`, push `2,1`; pop/emit `1`; pop/emit `2`; move to/push/pop `3`. Output is `[1,2,3]`. Each node is pushed and popped once: `O(n)` time, `O(h)` space.

Level-order BFS

Enqueue the root. At the beginning of each outer iteration, queue size equals exactly the number of nodes in the next level. Remove that many, collect their values, and enqueue children. FIFO order ensures levels do not mix. Time `O(n)`; frontier space is `O(w)`, maximum width `w`, which can be `O(n)` even when height is small.

Marking visited is unnecessary under the proper-tree contract. If shared/cyclic input is possible, use an identity-based visited set and redefine what traversal means.

Family 2: height, balance, diameter, and path sum

Height and balance

Define height before coding: this chapter uses number of nodes on the longest root-to-leaf path, so empty height is zero and leaf height is one. Height obeys `1 + max(leftHeight,rightHeight)`.

A naive balance check recomputes height at every node and can cost `O(n^2)` on a skewed tree. Instead, use a postorder helper that returns subtree height when balanced and sentinel `-1` when unbalanced. The contract lets failure propagate immediately. At a node, obtain both child results; reject if either is `-1` or their height difference exceeds one; otherwise return one plus their maximum. Every node is processed once: `O(n)` time, `O(h)` stack.

Diameter

Define diameter as number of edges on the longest path between any two nodes. Postorder height in nodes gives a through-node edge count `leftHeight + rightHeight`. Update a shared accumulator, then return this node's height. The invariant is that after processing a subtree, the accumulator holds the greatest diameter found anywhere in it and the return value is exactly its height.

For a root with left chain of two nodes and right leaf, child heights are `2` and `1`; through-root diameter is `3` edges. The longest path need not pass through the global root, so computing only root-left-height plus root-right-height is insufficient.

Path sum

For root-to-leaf target sum, subtract the current value from a `long` remainder. At a leaf, accept when remainder equals its value (or after subtracting, remainder is zero). A node with one null child is not a leaf. The helper explores each path independently, so no mutable global sum is needed. Time `O(n)`, stack `O(h)`.

Negative values invalidate pruning rules such as "stop when remainder becomes negative." Use that prune only when the value domain is nonnegative and explicitly guaranteed.

Family 3: lowest common ancestor

For a general binary tree where both target node identities are guaranteed present, use postorder:

- return the current node if it is `p` or `q`;
- recursively obtain evidence from left and right;
- if both are non-null, current is their lowest common ancestor;
- otherwise propagate whichever non-null node was found.

The helper returns null if neither target occurs, a target/ancestor if one side contains evidence, or the LCA once both sides contain evidence. Postorder ensures the first node combining two sides is lowest. Complexity is `O(n)` time and `O(h)` stack.

If either target may be absent, the basic result can wrongly return the one present node. Extend the return type with a found-count, or validate membership separately. Compare node identity, not equal values. In a BST, value ordering can reduce LCA to `O(h)` when keys are unique and both presence assumptions are defined.

Dry-run with root `6`, targets `2` and `8`: left returns `2`, right returns `8`, so root is LCA. For targets `2` and descendant `4`, the call at node `2` returns itself immediately, and ancestors propagate it; `2` is correctly the LCA.

Family 4: reconstruction and serialization

Reconstruct from preorder and inorder

Preorder supplies the next subtree root. Inorder splits values belonging to its left and right subtrees. With unique values, map each inorder value to its index. The helper receives an inorder interval; consume one preorder value as root, verify its mapped index lies inside, recursively build left interval then right interval.

For preorder `[3,9,20,15,7]` and inorder `[9,3,15,20,7]`, root `3` splits inorder into `[9]` and `[15,20,7]`. The next preorder value builds left node `9`. Next `20` splits the right interval into children `15` and `7`.

Each node is mapped and consumed once: `O(n)` time and `O(n)` map plus `O(h)` stack. Without a map, repeated inorder scanning can cost `O(n^2)`. Duplicate values make the traversals insufficient to determine a unique tree unless extra identity information or a deterministic convention is provided.

Serialization

Preorder with explicit null markers is self-delimiting: write value, then left, then right; write `#` for null. Deserialization consumes tokens under the same contract. For root `2` with children `1,3`, serialization is `2,1,#,#,3,#,#,`. Null markers are essential: preorder values alone cannot distinguish many shapes.

Round-trip properties are stronger than one expected string: `deserialize(serialize(tree))` should have the same structure and values, and serializing the restored tree should produce the canonical string. Define escaping if values are not plain integers, version the format if persisted, cap depth/input, and treat deserialization as an untrusted boundary.

Family 5: binary search tree operations

Global invariant

This chapter's BST contract uses unique keys:

> Every key in a node's left subtree is strictly less than the node key; every key in its right subtree is strictly greater.

Validation must enforce ancestor bounds, not only compare a node with immediate children. Tree `10` with left child `5` whose right child is `12` passes local parent-child checks but violates the root's left-subtree bound. Carry exclusive lower and upper `long` bounds; using `long` avoids overflow tricks around `Integer.MIN_VALUE` and `MAX_VALUE`.

Validation visits all nodes: `O(n)` time, `O(h)` stack. Inorder strict increase is another valid proof under the unique-key contract.

Search and insertion

At node `x`, if target is smaller, the global invariant proves the entire right subtree is impossible; go left. Greater goes right. Each comparison descends one level, so time is `O(h)`: `O(log n)` if balanced, `O(n)` if skewed.

Insertion follows the same path until a null child is found. The path bounds prove attaching the new key there preserves all ancestor constraints. Define duplicate policy: reject, ignore, count frequency, or consistently choose a side. The sample ignores duplicate insertion.

Deletion

Deletion has three structural cases:

1. no left child: replace node with right child;
2. no right child: replace node with left child;
3. two children: choose inorder successor, the minimum of the right subtree; copy its key into the node, then delete that successor from the right subtree.

The successor is the smallest key still greater than every left-subtree key. It has no left child, so its recursive deletion reduces to an easier case. For deleting `5` from BST keys `[5,3,7,6,8]`, successor `6` replaces `5`, and old `6` is removed below `7`.

Time is `O(h)`, recursive stack `O(h)`. If nodes have identity observed by clients, copying only a key can violate object-level expectations. A production API may transplant nodes instead, store immutable entries, or hide node identity.

Kth smallest

BST inorder is sorted. Iterative inorder with a counter returns when the kth node is popped. Time is `O(h+k)` in a balanced tree and `O(n)` worst case; stack `O(h)`. Repeated rank queries justify augmenting every node with subtree size, but then rotations/inserts/deletes must maintain a new invariant.

Family 6: tries and prefixes

A trie stores one edge per symbol. The node reached after consuming prefix `p` represents exactly all inserted words beginning with `p`. Insert walks/creates edges, then marks a terminal node. Exact search requires the terminal mark; prefix search requires only that the path exists. A pass count can answer how many inserted words share a prefix.

For words `car`, `card`, `care`, and `dog`, path `c-a-r` has pass count three and terminal true for `car`. `ca` is a prefix but not an exact word. Search/insert costs `O(L)` for word length `L`, independent of number of words, assuming constant-time edge access.

An array of 26 child references is fast for normalized lowercase English but wastes memory in sparse nodes and does not support arbitrary Unicode. A hash map is sparse but adds object/hash overhead and nondeterministic iteration unless ordered explicitly. Compressed radix trees, ternary search trees, finite-state structures, or database indexes may be better at scale. Always define normalization, case folding, locale, Unicode code point handling, maximum length, and duplicate semantics.

Complete Java 21 reference implementation

Compile and run with `java -ea TreesBstsTriesSde2`. The node algorithms assume a proper acyclic tree. The trie accepts only lowercase ASCII `a` through `z` so its representation contract is visible.

JAVA (continued 1/10)

```
import java.util.ArrayDeque;
import java.util.ArrayList;
import java.util.Deque;
import java.util.HashMap;
import java.util.List;
import java.util.Map;
import java.util.NoSuchElementException;

public final class TreesBstsTriesSde2 {
    private TreesBstsTriesSde2() {}

    public static final class Node {
        public int value;
        public Node left;
        public Node right;
        public Node(int value) { this.value = value; }
    }

    public static List<Integer> preorderRecursive(Node root) {
        List<Integer> answer = new ArrayList<>();
        preorder(root, answer);
        return answer;
    }

    private static void preorder(Node node, List<Integer> answer) {
        if (node == null) return;
        answer.add(node.value);
        preorder(node.left, answer);
        preorder(node.right, answer);
    }

    public static List<Integer> inorderIterative(Node root) {
        List<Integer> answer = new ArrayList<>();
        Deque<Node> stack = new ArrayDeque<>();
```

JAVA (continued 2/10)

```
        Node current = root;
        while (current != null || !stack.isEmpty()) {
            while (current != null) {
                stack.push(current);
                current = current.left;
            }
            current = stack.pop();
            answer.add(current.value);
            current = current.right;
        }
        return answer;
    }

    public static List<List<Integer>> levelOrder(Node root) {
        if (root == null) return List.of();
        List<List<Integer>> answer = new ArrayList<>();
        ArrayDeque<Node> queue = new ArrayDeque<>();
        queue.addLast(root);
        while (!queue.isEmpty()) {
            int levelSize = queue.size();
            List<Integer> level = new ArrayList<>(levelSize);
            for (int i = 0; i < levelSize; i++) {
                Node node = queue.removeFirst();
                level.add(node.value);
                if (node.left != null) queue.addLast(node.left);
                if (node.right != null) queue.addLast(node.right);
            }
            answer.add(level);
        }
        return answer;
    }

    public static int height(Node root) {
        return root == null ? 0 : 1 + Math.max(height(root.left), height(root.right));
```

JAVA (continued 3/10)

```
    }

    public static boolean isHeightBalanced(Node root) {
        return balancedHeight(root) >= 0;
    }

    private static int balancedHeight(Node node) {
        if (node == null) return 0;
        int left = balancedHeight(node.left);
        if (left < 0) return -1;
        int right = balancedHeight(node.right);
        if (right < 0 || Math.abs(left - right) > 1) return -1;
        return 1 + Math.max(left, right);
    }

    public static int diameterEdges(Node root) {
        int[] best = {0};
        diameterHeight(root, best);
        return best[0];
    }

    private static int diameterHeight(Node node, int[] best) {
        if (node == null) return 0;
        int left = diameterHeight(node.left, best);
        int right = diameterHeight(node.right, best);
        best[0] = Math.max(best[0], left + right);
        return 1 + Math.max(left, right);
    }

    public static boolean hasRootToLeafSum(Node root, long target) {
        if (root == null) return false;
        if (root.left == null && root.right == null) return target == root.value;
        long remaining = target - root.value;
        return hasRootToLeafSum(root.left, remaining)
```

JAVA (continued 4/10)

```
                || hasRootToLeafSum(root.right, remaining);
    }

    public static Node lowestCommonAncestor(Node root, Node p, Node q) {
        if (root == null || root == p || root == q) return root;
        Node left = lowestCommonAncestor(root.left, p, q);
        Node right = lowestCommonAncestor(root.right, p, q);
        if (left != null && right != null) return root;
        return left != null ? left : right;
    }

    public static Node buildFromPreorderInorder(int[] preorder, int[] inorder) {
        if (preorder == null || inorder == null || preorder.length != inorder.length) {
            throw new IllegalArgumentException("traversal lengths differ or are null");
        }
        Map<Integer, Integer> position = new HashMap<>();
        for (int i = 0; i < inorder.length; i++) {
            if (position.put(inorder[i], i) != null) {
                throw new IllegalArgumentException("values must be unique");
            }
        }
        int[] preorderIndex = {0};
        Node root = build(preorder, 0, inorder.length, preorderIndex, position);
        if (preorderIndex[0] != preorder.length) {
            throw new IllegalArgumentException("inconsistent traversals");
        }
        return root;
    }

    private static Node build(int[] preorder, int inLow, int inHigh,
                              int[] preorderIndex, Map<Integer, Integer> position) {
        if (inLow == inHigh) return null;
        if (preorderIndex[0] >= preorder.length) {
            throw new IllegalArgumentException("inconsistent traversals");
```

JAVA (continued 5/10)

```
        }
        int value = preorder[preorderIndex[0]++];
        Integer split = position.get(value);
        if (split == null || split < inLow || split >= inHigh) {
            throw new IllegalArgumentException("inconsistent traversals");
        }
        Node root = new Node(value);
        root.left = build(preorder, inLow, split, preorderIndex, position);
        root.right = build(preorder, split + 1, inHigh, preorderIndex, position);
        return root;
    }

    public static String serialize(Node root) {
        StringBuilder output = new StringBuilder();
        serialize(root, output);
        return output.toString();
    }

    private static void serialize(Node node, StringBuilder output) {
        if (node == null) {
            output.append("#,");
            return;
        }
        output.append(node.value).append(',');
        serialize(node.left, output);
        serialize(node.right, output);
    }

    public static Node deserialize(String encoded) {
        if (encoded == null || encoded.isEmpty()) {
            throw new IllegalArgumentException("empty encoding");
        }
        String[] tokens = encoded.split(",");
        int[] at = {0};
```

JAVA (continued 6/10)

```
        Node root = deserialize(tokens, at);
        if (at[0] != tokens.length) throw new IllegalArgumentException("extra tokens");
        return root;
    }

    private static Node deserialize(String[] tokens, int[] at) {
        if (at[0] == tokens.length) throw new IllegalArgumentException("truncated tree");
        String token = tokens[at[0]++];
        if (token.equals("#")) return null;
        final int value;
        try { value = Integer.parseInt(token); }
        catch (NumberFormatException error) {
            throw new IllegalArgumentException("bad node value", error);
        }
        Node node = new Node(value);
        node.left = deserialize(tokens, at);
        node.right = deserialize(tokens, at);
        return node;
    }

    public static boolean validBst(Node root) {
        return validBst(root, Long.MIN_VALUE, Long.MAX_VALUE);
    }

    private static boolean validBst(Node node, long lower, long upper) {
        if (node == null) return true;
        if (node.value <= lower || node.value >= upper) return false;
        return validBst(node.left, lower, node.value)
                && validBst(node.right, node.value, upper);
    }

    public static Node bstSearch(Node root, int target) {
        Node current = root;
```

JAVA (continued 7/10)

```
        while (current != null && current.value != target) {
            current = target < current.value ? current.left : current.right;
        }
        return current;
    }

    public static Node bstInsert(Node root, int value) {
        if (root == null) return new Node(value);
        Node current = root;
        while (true) {
            if (value == current.value) return root;
            if (value < current.value) {
                if (current.left == null) { current.left = new Node(value); return root; }
                current = current.left;
            } else {
                if (current.right == null) { current.right = new Node(value); return root; }
                current = current.right;
            }
        }
    }

    public static Node bstDelete(Node root, int value) {
        if (root == null) return null;
        if (value < root.value) root.left = bstDelete(root.left, value);
        else if (value > root.value) root.right = bstDelete(root.right, value);
        else {
            if (root.left == null) return root.right;
            if (root.right == null) return root.left;
            Node successor = root.right;
            while (successor.left != null) successor = successor.left;
            root.value = successor.value;
            root.right = bstDelete(root.right, successor.value);
        }
```

JAVA (continued 8/10)

```
        return root;
    }

    public static int kthSmallest(Node root, int k) {
        if (k <= 0) throw new IllegalArgumentException("k must be positive");
        Deque<Node> stack = new ArrayDeque<>();
        Node current = root;
        while (current != null || !stack.isEmpty()) {
            while (current != null) { stack.push(current); current = current.left; }
            current = stack.pop();
            if (--k == 0) return current.value;
            current = current.right;
        }
        throw new NoSuchElementException("k exceeds node count");
    }

    public static final class Trie {
        private static final class TrieNode {
            final TrieNode[] children = new TrieNode[26];
            int passCount;
            boolean terminal;
        }
        private final TrieNode root = new TrieNode();

        public void insert(String word) {
            requireWord(word);
            TrieNode node = root;
            node.passCount++;
            for (int i = 0; i < word.length(); i++) {
                int edge = edge(word.charAt(i));
                if (node.children[edge] == null) node.children[edge] = new TrieNode();
                node = node.children[edge];
                node.passCount++;
```

JAVA (continued 9/10)

```
            }
            node.terminal = true;
        }

        public boolean contains(String word) {
            TrieNode node = find(word);
            return node != null && node.terminal;
        }

        public boolean hasPrefix(String prefix) {
            return find(prefix) != null;
        }

        public int wordsWithPrefix(String prefix) {
            TrieNode node = find(prefix);
            return node == null ? 0 : node.passCount;
        }

        private TrieNode find(String text) {
            requireWord(text);
            TrieNode node = root;
            for (int i = 0; i < text.length(); i++) {
                node = node.children[edge(text.charAt(i))];
                if (node == null) return null;
            }
            return node;
        }

        private static int edge(char ch) {
            if (ch < 'a' || ch > 'z') {
                throw new IllegalArgumentException("only lowercase a-z supported");
            }
            return ch - 'a';
```

JAVA (continued 10/10)

```
        }

        private static void requireWord(String text) {
            if (text == null) throw new IllegalArgumentException("text is null");
        }
    }

    public static void main(String[] args) {
        Node root = buildFromPreorderInorder(
                new int[] {3, 9, 20, 15, 7}, new int[] {9, 3, 15, 20, 7});
        assert preorderRecursive(root).equals(List.of(3, 9, 20, 15, 7));
        assert inorderIterative(root).equals(List.of(9, 3, 15, 20, 7));
        assert levelOrder(root).equals(List.of(List.of(3), List.of(9, 20), List.of(15, 7)));
        assert height(root) == 3 && isHeightBalanced(root);
        assert diameterEdges(root) == 3;
        assert hasRootToLeafSum(root, 30);
        assert lowestCommonAncestor(root, root.right.left, root.right.right) == root.right;
        String encoded = serialize(root);
        assert serialize(deserialize(encoded)).equals(encoded);

        Node bst = null;
        for (int value : new int[] {5, 3, 7, 2, 4, 6, 8}) bst = bstInsert(bst, value);
        assert validBst(bst) && bstSearch(bst, 6).value == 6;
        assert kthSmallest(bst, 3) == 4;
        bst = bstDelete(bst, 5);
        assert validBst(bst) && inorderIterative(bst).equals(List.of(2, 3, 4, 6, 7, 8));

        Trie trie = new Trie();
        trie.insert("car"); trie.insert("card"); trie.insert("care"); trie.insert("dog");
        assert trie.contains("car") && !trie.contains("ca");
        assert trie.hasPrefix("ca") && trie.wordsWithPrefix("car") == 3;
    }
}
```

Complexity matrix

Operation	Time	Auxiliary space	Qualification
traversal / height / balance / diameter	`O(n)`	`O(h)` or BFS `O(w)`	proper tree assumed
LCA in general tree	`O(n)`	`O(h)`	basic helper assumes both identities exist
reconstruct traversals	`O(n)`	`O(n)` map + `O(h)` stack	unique, consistent values
serialize/deserialize	`O(n)`	`O(h)` stack + output	format and depth limits matter
BST search/insert/delete	`O(h)`	iterative `O(1)` or recursive `O(h)`	`h` may be `n` unless balanced
kth smallest	`O(h+k)` typical	`O(h)`	no subtree-size augmentation
trie operation	`O(L)`	insertion up to `O(L*alphabet slots)`	representation dominates memory

Avoid saying BST work is simply `O(log n)`. That bound requires a balancing guarantee such as AVL or red-black invariants; the basic tree above can become a linked list.

WATCH OUT | Edge cases and common mistakes

- Define height and diameter in nodes or edges; off-by-one answers often come from switching definitions.
- A null root may mean empty output, zero height, or false path existence depending on contract.
- Recursive state stored in mutable fields can leak across calls; prefer returned summaries or method-local holders.
- BFS queue size must be captured before processing a level because children are added during the loop.
- Balance should compute child heights once, not rescan subtrees.
- LCA must clarify target presence and identity versus value.
- Reconstruction from preorder/inorder needs unique identity or a duplicate policy.
- Null markers preserve serialization shape; delimiters and malformed-input handling must be explicit.
- BST validation requires ancestor bounds, not only parent-child comparisons.
- Duplicate BST keys require a consistent global policy.
- BST delete can change the root; callers must use the returned root.
- A trie prefix is not necessarily a complete stored word.
- Fixed child arrays trade memory for speed and only fit a declared alphabet.

TRY IT | Exercises with model checkpoints

Exercise 1: iterative preorder and postorder

Implement both without recursion.

Checkpoint: preorder can push right then left so left is processed first. Postorder may use two stacks or one stack with `(node,visited)` frames. State the frame invariant and `O(h)` versus potentially `O(n)` explicit storage.

Exercise 2: right-side view

Return the value visible from the right at every depth.

Checkpoint: BFS may record the final node of each captured level, or DFS may visit right before left and record the first value seen at a new depth. Both are `O(n)`; DFS stack risks skewed depth while BFS pays width.

Exercise 3: LCA with missing targets

Return an answer only if both node identities occur.

Checkpoint: return a record containing candidate and a two-bit/found count. Combine child counts plus current identity, and publish an LCA only at count two. Do not run two full membership traversals unless simplicity is preferred over one pass.

Exercise 4: iterative safe deserialization

Handle a persisted tree whose depth may be hostile.

Checkpoint: use explicit frames describing the next child slot, cap nodes/tokens/depth, reject extra/truncated tokens, and avoid recursive stack exhaustion. Include a version and integrity/authentication decision for untrusted storage.

Exercise 5: balanced BST guarantee

Explain how an AVL or red-black tree changes complexity.

Checkpoint: it maintains additional rotation/color/height invariants so height remains `O(log n)`. Operations pay rebalancing work but preserve worst-case lookup/update. Do not attempt to reimplement a production balanced map in an interview unless asked; focus on invariants.

Exercise 6: trie deletion

Delete one word while retaining shared prefixes.

Checkpoint: clear terminal only if present, decrement pass counts along the path, and remove a child link only when its pass count becomes zero. Define duplicate insertion: a boolean terminal cannot distinguish inserting the same word twice; use terminal frequency if multiset behavior is required.

Exercise 7: autocomplete top-k

Return the most frequent completions under a prefix.

Checkpoint: a naive subtree traversal is output/subtree dependent. Storing top-k summaries at prefix nodes speeds reads but increases update cost and consistency obligations. Define ranking ties, freshness, normalization, memory budget, and whether results are eventually consistent.

SDE-2 production follow-ups

How do you protect against skew? Use a balanced tree, iterative algorithms, depth limits, or a different index. Random input is not a guarantee. Monitor height or operation latency if the structure evolves from external keys.

Would you serialize raw object graphs? Prefer a versioned schema with validation, size/depth limits, and explicit fields. Never assume deserialized data is a proper tree. Avoid Java native serialization for untrusted boundaries; construction should enforce invariants before publication.

How does concurrency affect tree mutation? Insert/delete rewires several references while preserving a global order. Unsynchronized readers can see partial structure. Use confinement, immutable/persistent trees, copy-on-write snapshots for read-heavy workloads, or a proven concurrent index. `volatile` child fields alone do not make rotations or deletion atomic.

How do you choose trie representation? Measure alphabet size, sparsity, word count, prefix distribution, latency, and update frequency. Arrays optimize predictable small alphabets; maps optimize sparse edges; compressed tries reduce unary chains. Normalize text consistently at ingestion and query.

What tests establish confidence? For traversals, compare recursive and iterative outputs. For serialization, use round-trip properties. For BSTs, after random operations verify strict inorder order and a reference set. For tries, compare prefix counts against a simple list oracle. Include empty, singleton, skewed, duplicate, minimum/maximum key, malformed encoding, and unsupported-symbol inputs.

Final readiness checklist

- I choose preorder, inorder, postorder, or BFS from information flow.
- Every helper has a return contract and an explicit empty-tree identity.
- I count height `h` and frontier width `w`, not only node count.
- My LCA answer states presence and identity assumptions.
- Reconstruction and serialization preserve shape, not merely values.
- BST validation carries ancestor bounds and declares duplicate policy.
- I say `O(h)`, then explain when height is logarithmic or linear.
- Trie alphabet, normalization, terminal state, prefix counts, and memory trade-offs are explicit.

Tree mastery is disciplined information flow: children return exactly what parents need, ordering lets a branch be eliminated only when a global invariant permits it, and representation choices remain visible in both correctness and production cost.

SDE-2 CORE CHAPTER

Chapter 3 - Essential Tree and BST Pattern Clinics

Tree interviews often hide two distinct questions inside one traversal: what a subtree returns to its parent, and what complete answer may pass through that subtree. Maximum path sum makes that distinction explicit. BST successor makes ancestor bounds and ordering equally concrete.

Clinic 1: maximum path sum in a binary tree

A path may start and end at any nodes but cannot reuse a node. At each node, distinguish:

- **return value:** the best downward path that the parent may extend;
- **complete candidate:** the node plus the best nonnegative contribution from both children.

The parent can extend at most one child branch because taking both would create a fork rather than a path. The global answer may use both branches because it ends the connection at the current node.

```text
down(node) = node.value + max(0, down(left), down(right))
through(node) = node.value + max(0, down(left)) + max(0, down(right))
```

Initialize the global best below every possible node value, not to zero. Otherwise an all-negative tree incorrectly returns an empty path even though the contract requires at least one node.

For `-10` with children `9` and `20`, where `20` has children `15` and `7`, the best complete candidate is `15 + 20 + 7 = 42`. The downward value returned from `20` is only `20 + 15 = 35`.

Clinic 2: inorder successor in a BST

Assume distinct keys. The successor is the smallest key greater than the target.

- If the target has a right subtree, the successor is the leftmost node in that subtree.
- Otherwise, it is the lowest ancestor for which the target lies in the ancestor's left subtree.

During search, moving left records the current node as a candidate successor; moving right discards the current node because its key is smaller. This gives O(h) time and O(1) auxiliary space without parent links.

If duplicate keys are allowed, the ordering policy and whether the target is a key or a node identity must be stated first. A value-only API cannot distinguish equal nodes.

Runnable Java 21 clinic

JAVA (continued 1/3)

```java
import java.util.NoSuchElementException;

public final class TreeCoverageClinic {
    private TreeCoverageClinic() {
    }

    public static final class Node {
        private final int value;
        private Node left;
        private Node right;

        public Node(int value) {
            this.value = value;
        }
    }

    public static long maximumPathSum(Node root) {
        if (root == null) {
            throw new IllegalArgumentException("root must be nonnull");
        }
        long[] best = {Long.MIN_VALUE};
        maximumDownwardContribution(root, best);
        return best[0];
    }

    private static long maximumDownwardContribution(Node node, long[] best) {
```

JAVA (continued 2/3)

```java
        if (node == null) {
            return 0;
        }
        long left = Math.max(0, maximumDownwardContribution(node.left, best));
        long right = Math.max(0, maximumDownwardContribution(node.right, best));
        best[0] = Math.max(best[0], node.value + left + right);
        return node.value + Math.max(left, right);
    }

    public static Node inorderSuccessor(Node root, int target) {
        Node current = root;
        Node successor = null;
        while (current != null && current.value != target) {
            if (target < current.value) {
                successor = current;
                current = current.left;
            } else {
                current = current.right;
            }
        }
        if (current == null) {
            throw new NoSuchElementException("target is not in the BST");
        }
        if (current.right != null) {
            successor = current.right;
```

JAVA (continued 3/3)

```
            while (successor.left != null) {
                successor = successor.left;
            }
        }
        return successor;
    }

    public static void main(String[] args) {
        Node root = new Node(-10);
        root.left = new Node(9);
        root.right = new Node(20);
        root.right.left = new Node(15);
        root.right.right = new Node(7);
        assert maximumPathSum(root) == 42;

        Node bst = new Node(20);
        bst.left = new Node(10);
        bst.right = new Node(30);
        bst.left.left = new Node(5);
        bst.left.right = new Node(15);
        assert inorderSuccessor(bst, 15).value == 20;
        assert inorderSuccessor(bst, 30) == null;
        System.out.println("PASS essential tree clinics");
    }
}
```

Expected output with assertions enabled:

TEXT

```
PASS essential tree clinics
```

Interviewer follow-up chain with model answers

Interviewer: Why can the recursive function not return the full through-node path?

Candidate: A parent could attach to only one endpoint. Returning a path that already uses both child branches would give the current node degree three when the parent attaches, which is not a simple path.

Interviewer: What is the auxiliary space?

Candidate: O(h) active call frames, where h is tree height. It is O(log n) for a balanced tree and O(n) for a chain, which can overflow the Java stack on adversarial depth.

Interviewer: How would successor change with parent pointers?

Candidate: If there is no right subtree, walk upward until leaving an ancestor's left edge. That uses O(1) extra space and O(h) time without starting again from the root.

SDE-2 CORE CHAPTER

Chapter 4 - Realistic Tree, BST, and Trie Interview Rounds

Round 1: validate a binary search tree

Prompt

Return whether a binary tree satisfies the strict BST invariant. Values are Java `int` values.

Candidate clarification

> Are duplicate keys permitted, and if so, on which side?

The interviewer says duplicates are invalid.

Correct answer

Carry the valid open interval from ancestors. Use `long` bounds so all `int` values are representable without special casing.

JAVA

```java
static boolean isValidBst(TreeNode root) {
    return valid(root, Long.MIN_VALUE, Long.MAX_VALUE);
}

static boolean valid(TreeNode node, long lower, long upper) {
    if (node == null) {
        return true;
    }
    if (node.value <= lower || node.value >= upper) {
        return false;
    }
    return valid(node.left, lower, node.value)
            && valid(node.right, node.value, upper);
}
```

Invariant: every node reached by a call must lie strictly inside the interval implied by all ancestors. The left child receives a tighter upper bound; the right child receives a tighter lower bound.

Complexity: O(n) time; O(h) stack space.

Failure mode: checking `node.left.value < node.value` and `node.right.value > node.value` misses a value in the right subtree that is smaller than the root.

Round 2: lowest common ancestor in a general binary tree

Prompt

Return the lowest node whose subtree contains both target node references. Both targets are guaranteed present.

Model answer

JAVA

```java
static TreeNode lowestCommonAncestor(TreeNode node, TreeNode first, TreeNode second) {
    if (node == null || node == first || node == second) {
        return node;
    }
    TreeNode left = lowestCommonAncestor(node.left, first, second);
    TreeNode right = lowestCommonAncestor(node.right, first, second);
    if (left != null && right != null) {
        return node;
    }
    return left != null ? left : right;
}
```

Contract: a call returns a target found in its subtree, the LCA if both are found, or null if neither is found. When left and right are both non-null, the targets split across the child subtrees, so the current node is lowest.

Follow-up answers

What if a target may be absent? The simple method can return the present target and falsely imply success. Return a result containing candidate plus found-count, and accept only count two.

What if this is a BST? Ordering can guide both targets left or right, giving O(h) search without exploring both subtrees.

Round 3: design autocomplete with a trie

Prompt

Support inserting words, testing complete words, testing prefixes, and returning up to k suggestions for a prefix.

Candidate design

The basic trie stores a child map and terminal flag. To provide deterministic lexicographic suggestions, either use ordered children or sort child keys during collection. For high-query systems, cache top suggestions at nodes and define update consistency.

JAVA

```java
static final class TrieNode {
    final Map<Character, TrieNode> children = new TreeMap<>();
    boolean terminal;
}

static void insert(TrieNode root, String word) {
    TrieNode current = root;
    for (int i = 0; i < word.length(); i++) {
        current = current.children.computeIfAbsent(word.charAt(i), ignored -> new TrieNode());
    }
    current.terminal = true;
}
```

Follow-up answers

Unicode? `char` traverses UTF-16 code units, not complete user-perceived characters. Define normalization and symbol unit; code-point traversal may be required.

Complexity? Basic insert and prefix navigation are $O(L)$ child lookups. Suggestion generation is output-sensitive and may visit a large subtree. `TreeMap` adds logarithmic child lookup; fixed alphabets can use arrays with different memory trade-offs.

How do rankings work? Store word frequencies and use cached top-k lists, a heap during subtree search, or a separate search index. Define freshness, memory, and update costs.

Closing answer pattern

State node meaning, null base case, returned subtree contract, traversal order, height-dependent space, identity versus value, duplicate/order policy, and skewed-tree behavior.

SDE-2 CORE CHAPTER

Chapter 5 - Range-Query Trees and AVL Rotations

Prefix sums answer immutable range sums beautifully. The moment values change between queries, the prefix array becomes stale. Fenwick and segment trees solve that update/query tension. AVL trees solve a related but different problem: preserving ordered-search height after insertions.

These structures are easier when learned as maintained invariants, not as formulas to memorize. Complete Java implementations and randomized checks are in `TreeInterviewChecks.java`.

Start with the workload

Suppose an array receives point updates and half-open range-sum queries `[left, right)`.

Structure	Build	Point update	Range sum	Extra space	Best fit
scan array	none	`O(1)`	`O(n)`	`O(1)`	very few queries
prefix sums	`O(n)`	`O(n)` rebuild	`O(1)`	`O(n)`	immutable data
Fenwick tree	`O(n log n)` in companion	`O(log n)`	`O(log n)`	`O(n)`	prefix-compatible aggregate, compact code
segment tree	`O(n)`	`O(log n)`	`O(log n)`	`O(n)`	flexible associative aggregates/ranges

The companion uses `long` aggregate storage even though input values are `int`. A range sum can overflow before assignment if accumulation stays in `int`.

Fenwick tree: buckets defined by the lowest set bit

Fenwick storage is internally one-based. At internal index `i`, the value `i & -i` gives the width of the suffix interval summarized by `tree[i]`.

TEXT

```
internal index (binary)   lowbit   covered internal indexes
1  (001)                    1      [1,1]
2  (010)                    2      [1,2]
3  (011)                    1      [3,3]
4  (100)                    4      [1,4]
5  (101)                    1      [5,5]
6  (110)                    2      [5,6]
8 (1000)                    8      [1,8]
```

One-based indexing is an internal implementation tool. The public API remains zero-based:

- `add(index, delta)` changes one array position;

- `prefixSum(rightExclusive)` sums `[0, rightExclusive)`; and
- `rangeSum(left, rightExclusive)` subtracts two prefixes.

Update trace

For a length-eight tree, updating public index `2` starts at internal index `3`:

```text
3  (0011) + lowbit 1 -> 4
4  (0100) + lowbit 4 -> 8
8  (1000) + lowbit 8 -> stop beyond length
```

Those nodes are exactly the summaries whose covered interval contains the changed element.

Prefix trace

To query the first seven public elements, start with internal `7`:

```text
7 (0111) contributes [7,7]; subtract lowbit 1 -> 6
6 (0110) contributes [5,6]; subtract lowbit 2 -> 4
4 (0100) contributes [1,4]; subtract lowbit 4 -> 0
```

The intervals are disjoint and cover `[1,7]`, so each tree cell is added once. Both operations take `O(log n)` because each step clears or adds a set bit.

Fenwick boundary conditions

An empty structure can represent length zero, but no point update is valid. Prefix endpoint zero is valid and returns zero. A half-open empty range such as `[3,3)` is valid. Keep public and internal indexes distinct in variable names; most bugs are accidental mixing.

Segment tree: store summaries of intervals

The companion uses an iterative segment tree. Leaves begin at a power-of-two base, and each parent stores the sum of its two children.

For values `[2, -1, 4, 3, 5]`, the next power of two is eight:

```text
                    [0,8) sum=13
                  /                \
          [0,4) sum=8              [4,8) sum=5
           /       \                /       \
      [0,2)=1      [2,4)=7      [4,6)=5      [6,8)=0
       /   \        /   \        /   \        /   \
      2    -1       4     3     5    0        0     0
```

Padding leaves contribute the identity value zero. For minimum queries, the identity would be positive infinity; for maximum, negative infinity. The combine operation must be associative so a range can be split and regrouped safely.

Point replacement

Setting index 2 from `4` to `0` changes its leaf. Recompute each ancestor on the path to the root. Only `O(log n)` nodes change.

Iterative half-open query

The query maintains two accumulators while leaf pointers move inward:

TEXT

```
[left + base, right + base)
 odd left pointer  -> include it, then increment
 odd right pointer -> decrement, then include it
 shift both pointers to parents
```

The left and right accumulators matter for noncommutative associative operations: left fragments must remain in left-to-right order. Sums are commutative, but the implementation pattern should still preserve order.

Fenwick or segment tree?

Choose Fenwick when prefix subtraction can recover the desired range and compact code matters. Choose a segment tree when the query needs a more general associative summary, such as minimum, maximum, greatest common divisor, or a custom node containing multiple fields.

Neither structure automatically supports every update. Range updates plus range queries may require a lazy segment tree or a paired Fenwick technique. State the exact update and query operations before choosing.

WHY IT WORKS | AVL trees: ordered search with a height invariant

An ordinary BST can become a chain after sorted insertion, making search and insert `O(n)`. An AVL tree maintains, for every node:

TEXT

```
balance = height(left) - height(right)
balance must be -1, 0, or 1
height = 1 + max(height(left), height(right))
```

After a normal BST insertion, update heights while returning toward the root. The first unbalanced ancestor determines a rotation case.

LL: one right rotation

TEXT

```
before               after rotateRight(30)
      30                       20
     /                        /  \
   20                       10    30
  /
10
```

RR: one left rotation

TEXT

```
10                              20
  \                            /  \
   20      rotateLeft(10)     10   30
     \
      30
```

LR: rotate the child, then the ancestor

TEXT

```
    30             30                 20
   /              /                  /  \
 10      ->      20        ->       10   30
   \            /
   20          10
 rotateLeft(10)       rotateRight(30)
```

RL: the mirror image

Rotate the right child right, then the ancestor left. Naming the case from the path from the unbalanced node to the inserted key-right then left-helps avoid memorized diagrams.

The transferred subtree

A rotation must preserve BST order. During a right rotation, the promoted node's right subtree moves to the old root's left:

TEXT

```
       top                promoted
       /                  /      \
 promoted       ->       A        top
  /   \                             /
 A     B                           B
```

Every key in B lies between the promoted key and top key, so it belongs in exactly that transferred position. Update the demoted node's height first, then the promoted node's height.

The companion implements set semantics: duplicate keys are ignored. A multiset AVL would need a count field or a tie-breaking identity.

Recursion and production boundaries

AVL height is logarithmic, so recursive insertion depth is logarithmic when the invariant already holds. The validation DFS and SCC code elsewhere can still overflow on arbitrary deep inputs. In production Java, prefer established ordered collections unless an interview explicitly asks for internals.

Java's `TreeMap` and `TreeSet` are balanced-tree collections, but their exact balancing strategy is an implementation detail distinct from this educational AVL.

Edge-case matrix

Case	Expected handling	Frequent failure
Fenwick public index zero	convert to internal one before update	infinite loop at internal zero
prefix endpoint `n`	valid	rejecting the complete prefix
empty range `[i,i)`	sum is zero	using inclusive/exclusive endpoints inconsistently
negative values/deltas	valid for sum structures	assuming summaries only increase
cumulative overflow	aggregate in `long` and state bounds	storing tree nodes in `int`
segment length not power of two	pad to next power with identity	reading padding as data
empty segment tree	empty range valid; point set invalid	building zero-length backing storage
duplicate AVL key	follow stated set/multiset policy	rotating after a non-insertion
LL/RR versus LR/RL	inspect child direction before rotating	applying a single rotation to a zig-zag
transferred AVL subtree	reconnect before height updates	losing nodes or breaking order
stale height	recompute bottom-up after links change	accepting a tree that looks balanced locally

Real interview follow-up round

Interviewer: Why can a Fenwick tree answer range sum with two prefix queries?

Candidate: Addition has an inverse. The prefix `[0,right)` contains `[0,left)` plus `[left,right)`, so subtracting removes the earlier part. That reasoning does not transfer to every aggregate; minimum has no corresponding inverse.

Interviewer: Can you use a Fenwick tree for range minimum?

Candidate: Not with the ordinary two-prefix subtraction design. Restricted monotonic updates have specialized variants, but a general point-update/range-minimum contract is more naturally handled by a segment tree.

Interviewer: Why is segment-tree storage commonly about `4n` recursively or `2 * powerOfTwo` iteratively?

Candidate: The logical tree pads leaves to a complete binary-tree shape. The exact allocation depends on layout; both bounds ensure parent/child positions exist. The companion uses `2 * nextPowerOfTwo`, which also makes leaf indexing direct.

Interviewer: Does one AVL rotation change inorder order?

Candidate: No. It changes parent-child structure while preserving the `A < promoted < B < top` ordering. That preservation is the core rotation proof.

Interviewer: How would you validate these implementations beyond examples?

Candidate: I would run random point replacements and range queries against a plain array, updating the Fenwick tree by delta and the segment tree by replacement. For AVL, I would compare inorder output and size with `TreeSet` after every random insertion and independently verify stored heights and balance factors. Those differential tests run in the companion.

Run the verified companion

BASH

```
javac -Xlint:all -Werror TreeInterviewChecks.java
java TreeInterviewChecks
```

Expected final line:

TEXT

```
PASS 18 tree checks
```

Use the **Arrays and Array Patterns** volume first for prefix sums and half-open ranges. Continue to advanced range-update structures only when a problem's workload requires them; do not replace a simple immutable prefix array with a tree by reflex.

SDE-2 CORE CHAPTER

Chapter 6 - Constant-Space Traversal and Repeated Ancestor Queries

Why this chapter exists

The tree material so far solves each problem once. This chapter covers the two follow-ups that arrive when an interviewer pushes on *resources* rather than correctness:

- **"Can you do that traversal in O(1) extra space?"** Recursion costs O(h) stack; an explicit stack costs O(h) heap. Morris traversal costs neither, and the trick that makes it work is worth understanding.
- **"Now answer a million LCA queries."** The postorder LCA from chapter 1 is O(n) per query, so a million queries on a million-node tree is 10^12 operations. Binary lifting preprocesses once and answers each query in O(log n).

Both are recognizably "second-half of the interview" questions. Neither is exotic, and each has a specific condition under which it is the wrong choice - which is the part worth being able to say.

Part 1: Morris traversal

The idea

An inorder traversal must, after finishing a left subtree, return to the parent. Recursion and an explicit stack both *remember* the parent. Morris traversal instead **temporarily writes the way back into the tree itself**, using the null right pointers that leaf nodes already have.

For any node with a left child, its inorder predecessor is the rightmost node of that left subtree - and that node's right pointer is null. Point it at the current node. Now, after the left subtree is exhausted, following right pointers naturally arrives back at the current node. That temporary pointer is called a **thread**.

TEXT

```
        4
       / \
      2   6
     / \
    1   3

Processing 4: predecessor is 3 (rightmost of the left subtree).
Thread 3.right -> 4.

        4
       / \
      2   6
     / \
    1   3 ....thread back to 4

Descend left. After 1, 2, 3 are visited, following 3.right returns to 4.
The thread is then removed and the tree is restored.
```

Implementation

JAVA

```java
static List<Integer> morrisInorder(TreeNode root) {
    List<Integer> out = new ArrayList<>();
    TreeNode current = root;

    while (current != null) {
        if (current.left == null) {
            out.add(current.val);          // no left subtree: visit and go right
            current = current.right;
            continue;
        }

        // Find the inorder predecessor: rightmost node of the left subtree.
        // Stop at `current` too, because a thread may already point back.
        TreeNode predecessor = current.left;
        while (predecessor.right != null && predecessor.right != current) {
            predecessor = predecessor.right;
        }

        if (predecessor.right == null) {
            predecessor.right = current;   // first visit: thread and descend
            current = current.left;
        } else {
            predecessor.right = null;      // second visit: unthread and visit
            out.add(current.val);
            current = current.right;
        }
    }
    return out;
}
```

Each node is reached at most twice, and each edge is walked at most a constant number of times, so the traversal remains **O(n) time with O(1) extra space** - the output list aside.

The `predecessor.right != current` condition in the inner loop is what makes the second visit detectable. Without it, the search for the predecessor follows the thread it created and loops forever. That single condition is the most commonly omitted line.

Preorder is one line different

Move the visit to the moment the thread is created rather than removed:

JAVA

```java
if (predecessor.right == null) {
    out.add(current.val);       // preorder: visit on the way down
    predecessor.right = current;
    current = current.left;
} else {
    predecessor.right = null;   // inorder would visit here instead
    current = current.right;
}
```

Postorder with Morris is possible but requires reversing the right-edge chain on each retreat. It is significantly more code, rarely asked, and reasonable to name without implementing.

When Morris is the wrong answer

This is the part that separates understanding from recitation.

It mutates the tree during traversal. The structure is restored by the end, but at any intermediate moment the tree contains threads. That means:

- **It is not safe under concurrent readers.** Another thread traversing simultaneously will follow a thread pointer and see a corrupted structure. An explicit stack is read-only and safe.
- **It requires mutable nodes.** If `TreeNode.right` is final, or the tree is a shared immutable structure, Morris is simply unavailable.
- **An exception mid-traversal leaves the tree threaded.** There is no natural try/finally repair, because the repair state is spread across the tree.

So the honest answer to "can you traverse in O(1) space" is: yes, with Morris, and here is the trade - the tree is temporarily modified, so it is unsuitable for shared or concurrently-read structures. That framing is what an interviewer is listening for. In most production code, the O(h) stack is a price worth paying for a read-only traversal.

Part 2: Binary lifting for repeated LCA

Why the O(n) LCA is not enough

The recursive postorder LCA visits every node, so it is O(n) per query. That is optimal for a single query. For `q` queries it is O(n*q), and interviewers escalate to exactly this point.

Binary lifting preprocesses the tree once in O(n log n) and answers each query in O(log n). The idea rests on one observation: **every integer is a sum of powers of two**, so any upward jump of `k` levels can be made in at most log(k) jumps of power-of-two size.

The ancestor table

`up[k][v]` is the 2^k-th ancestor of `v`, or a sentinel when that ancestor does not exist.

```text
up[0][v] = parent(v)
up[k][v] = up[k-1][ up[k-1][v] ]     jump halfway, then halfway again
```

That recurrence is the whole technique. Building the table is a DFS to record parents and depths, then a doubling loop.

JAVA (continued 1/2)

```java
public final class LcaBinaryLifting {
    private static final int NONE = -1;

    private final int[][] up;      // up[k][v] = 2^k-th ancestor of v
    private final int[] depth;
    private final int levels;

    public LcaBinaryLifting(List<List<Integer>> children, int root) {
        int n = children.size();
        levels = Math.max(1, 32 - Integer.numberOfLeadingZeros(Math.max(1, n)));
        up = new int[levels][n];
        depth = new int[n];
        for (int[] row : up) {
            Arrays.fill(row, NONE);
        }

        // Iterative DFS: recursion would overflow on a degenerate chain.
        Deque<int[]> stack = new ArrayDeque<>();
        stack.push(new int[]{root, NONE, 0});
        while (!stack.isEmpty()) {
            int[] frame = stack.pop();
            int node = frame[0];
            up[0][node] = frame[1];
            depth[node] = frame[2];
            for (int child : children.get(node)) {
                stack.push(new int[]{child, node, frame[2] + 1});
            }
        }

        for (int k = 1; k < levels; k++) {
            for (int v = 0; v < n; v++) {
                int middle = up[k - 1][v];
                up[k][v] = middle == NONE ? NONE : up[k - 1][middle];
            }
        }
```

JAVA (continued 2/2)

```
    }

    /** Lowest common ancestor of a and b in O(log n). */
    public int lca(int a, int b) {
        if (depth[a] < depth[b]) {
            int swap = a;
            a = b;
            b = swap;                       // a is now the deeper node
        }

        int difference = depth[a] - depth[b];
        for (int k = 0; k < levels; k++) {  // lift a to b's depth
            if ((difference >> k & 1) == 1) {
                a = up[k][a];
            }
        }
        if (a == b) {
            return a;                       // b was an ancestor of a
        }

        // Descend from the largest jump: move both up while they stay distinct.
        for (int k = levels - 1; k >= 0; k--) {
            if (up[k][a] != up[k][b]) {
                a = up[k][a];
                b = up[k][b];
            }
        }
        return up[0][a];                    // parents are now equal
    }

    /** Distance in edges, a direct application of the LCA. */
    public int distance(int a, int b) {
        return depth[a] + depth[b] - 2 * depth[lca(a, b)];
    }
}
```

Three details carry the correctness:

The equal-depth check before the second loop. If `b` is an ancestor of `a`, lifting `a` to `b`'s depth lands exactly on `b`. The second loop would then never move and return `up[0][b]`, which is one level too high. Returning early is not an optimization; it is required.

The second loop descends from the largest `k`. It moves both nodes up only while their `2^k`-th ancestors *differ*, which keeps them strictly below the LCA. After the loop they are children of it, so `up[0][a]` is the answer. Ascending `k` order does not work: a small jump taken first can overshoot past the LCA and there is no way back.

The DFS is iterative. A recursive build overflows on a degenerate chain - exactly the shape a hostile test uses - and this is a preprocessing step, so the depth is the tree's full height.

Costs, and when not to bother

Approach	Preprocess	Per query	Extra space
Recursive postorder	none	O(n)	O(h) stack
Binary lifting	O(n log n)	O(log n)	O(n log n)
Euler tour + sparse table	O(n log n)	**O(1)**	O(n log n)
Tarjan offline (Union-Find)	O(n + q) total	amortized	O(n)

Do not preprocess for a single query. With one LCA to answer, the recursive version wins on every axis - simpler, less memory, no build cost. Binary lifting pays for itself once `q` is comparable to `log n` and clearly once it is large.

If all queries are known in advance, Tarjan's offline algorithm with Union-Find is O(n + q) overall and beats both - worth naming as the answer to "what if I gave you every query up front", and it connects directly to the Union-Find material in the graphs volume.

Binary lifting also generalizes past LCA: the same table answers "the k-th ancestor of v" in O(log k), and with an extra table storing aggregates it answers max, min, or sum along the path between two nodes in O(log n).

WATCH OUT | Edge cases and common mistakes

- Omitting `predecessor.right != current` in Morris, so the predecessor search follows its own thread and loops forever.
- Using Morris on a tree that other threads read concurrently, or on immutable nodes.
- Assuming an exception during Morris leaves the tree intact; threads persist.
- Placing the Morris visit in the wrong branch, silently producing preorder instead of inorder.
- Claiming Morris is O(1) space while returning an O(n) result list; the bound covers auxiliary space only.
- Omitting the `a == b` early return in binary lifting, returning the LCA's parent when one node is an ancestor of the other.
- Iterating `k` upward in the descent loop, overshooting past the LCA.
- Building the ancestor table with recursion and overflowing on a degenerate chain.
- Sizing `levels` too small; it must satisfy `2^levels > n`.
- Forgetting to propagate the NONE sentinel, so a missing ancestor indexes `up[k-1][-1]`.
- Preprocessing for a single query, where the O(n) recursion is strictly better.
- Reaching for binary lifting when every query is known up front and Tarjan's offline method is O(n + q).

TRY IT | Interview questions and model answers

Traverse a binary tree inorder in O(1) extra space.

Morris traversal. For each node with a left child, find its inorder predecessor - the rightmost node of the left subtree - and point that node's null right pointer back at the current node. Descend left; when the thread is followed back, remove it and visit. Each node is reached at most twice, so it is O(n) time and O(1) auxiliary space.

What is the catch with Morris?

It mutates the tree while running. The structure is restored by the end, but intermediate states contain threads, so it is unsafe if another thread reads concurrently, impossible if nodes are immutable, and leaves the tree corrupted if an exception escapes mid-traversal. In most production code an O(h) stack is worth paying for a read-only traversal.

Your postorder LCA is O(n). I need a million queries.

Binary lifting. Preprocess `up[k][v]`, the 2^k-th ancestor, with `up[k][v] = up[k-1][up[k-1][v]]`, in O(n log n). Per query, lift the deeper node to equal depth using the binary representation of the depth difference, return early if they now coincide, then move both up from the largest jump while their ancestors differ. Answer is the common parent. O(log n) per query.

Why does the descent loop go from the largest k down?

Because it moves both nodes only while their `2^k`-th ancestors differ, which guarantees they stay strictly below the LCA. Starting from small jumps could take a step that lands at or above the LCA, and there is no mechanism to undo it.

Why the early return when the nodes coincide after lifting?

That case means `b` was an ancestor of `a`. The descent loop would find all ancestors equal, never move, and return `up[0][b]` - the parent of the true answer. The early return is required for correctness, not speed.

What if I gave you all the queries in advance?

Tarjan's offline LCA with Union-Find, which is O(n + q) overall and beats binary lifting's O(n log n + q log n). It works by a single DFS that unions each finished subtree into its parent and answers queries when the second endpoint is reached.

TRY IT | Exercises

1. **Foundation:** Trace Morris inorder on a three-node tree, drawing the thread at each step.
2. **Foundation:** Remove `predecessor.right != current` and describe exactly why the traversal never ends.
3. **Interview Core:** Implement Morris inorder and preorder, and verify both against recursive traversals over a thousand random trees.
4. **Interview Core:** Assert the tree is structurally unchanged after Morris completes, then throw from inside the loop and assert it is *not*.
5. **Interview Core:** Build the binary-lifting table and answer LCA queries. Verify against the recursive postorder version on random trees.
6. **Interview Core:** Remove the `a == b` early return and find the input where it returns the wrong node.
7. **Interview Core:** Reverse the descent loop to ascending `k` and construct the tree where it overshoots.
8. **SDE-2 Follow-up:** Measure the query count at which binary lifting overtakes the recursive LCA on a 10^5-node tree.
9. **SDE-2 Follow-up:** Extend the table to answer "maximum edge weight on the path between a and b" in O(log n).
10. **Challenge:** Implement Tarjan's offline LCA with Union-Find and compare against binary lifting for 10^6 known queries.

RECAP | Chapter summary

Both techniques answer resource follow-ups rather than correctness ones. Morris traversal achieves O(1) auxiliary space by writing the return path into the tree's own null right pointers, threading each node to its inorder predecessor and removing the thread on the way back; the predecessor search must stop at the current node or it follows its own thread forever. Its cost is that the tree is mutated during the walk, which rules it out for concurrently-read or immutable structures and leaves damage if an exception escapes - so the complete answer names the trade rather than just the space bound. Binary lifting turns O(n)-per-query LCA into O(log n) by tabulating 2^k-th ancestors through the doubling recurrence, requiring an early return when one node is an ancestor of the other and a descent from the largest jump downward so the pair never overshoots. It is not worth building for a single query, and if every query is known in advance Tarjan's offline method with Union-Find is better still.

TRY IT | Revision checklist

- [ ] I can explain what a Morris thread is and which pointer it reuses.
- [ ] I know why the predecessor search must stop at the current node.
- [ ] I can convert Morris inorder to preorder by moving one line.
- [ ] I can state the three situations where Morris is unusable.
- [ ] I can write the binary-lifting recurrence from memory.
- [ ] I know why the early equal-node return is required, not an optimization.
- [ ] I can explain why the descent loop runs from the largest k downward.
- [ ] I build the ancestor table iteratively and can say why.
- [ ] I can compute the query count at which preprocessing pays for itself.
- [ ] I know Tarjan's offline LCA is better when all queries are known up front.

SDE-2 CORE CHAPTER

Chapter 7 - Trees, BSTs, and Tries Practice Lab

1. Define root, leaf, depth, height, ancestor, and subtree on one example.
2. Write preorder, inorder, postorder, and level order for a seven-node tree.
3. Explain why inorder is sorted only for a valid BST.
4. Debug a height method whose null base returns 1 while height counts nodes.
5. Debug BST validation that checks only direct children.
6. Implement iterative preorder and postorder.
7. Return whether a tree is height-balanced in O(n), not O(n squared).
8. Compute tree diameter while defining edges versus nodes.
9. Serialize and deserialize a general binary tree with null markers.
10. Return the kth smallest BST value without materializing all values.
11. Implement trie deletion without removing nodes required by another word.
12. Return a right-side view using DFS or BFS.
13. Explain recursion risk for a million-node chain.
14. Compare a BST, `TreeMap`, and trie for prefix queries.
15. Design LCA queries when the tree is static and queries are frequent.
16. **Interview Core:** Return the maximum path sum when a path may start and end anywhere. Separate the value returned to the parent from the complete through-node candidate.
17. **Interview Core:** Find a BST node's inorder successor without parent pointers. State a distinct-key policy and the missing-target behavior.

Range-query and balancing lab

18 **Foundation:** For internal Fenwick index 12 (`1100`), compute its low bit and the interval summarized by that cell.

19 **Interview Core:** Implement a zero-based Fenwick API with point delta, half-open prefix, and half-open range sum. Test endpoint zero, endpoint `n`, and an empty range.

20 **Interview Core:** Implement an iterative point-replacement/range-sum segment tree using a power-of-two leaf base and `long` summaries.

21 **Interview Core:** Draw and execute LL, RR, LR, and RL AVL insertions. For each rotation, identify the transferred subtree and height-update order.

22 **SDE-2 Follow-up:** Differential-test Fenwick and segment trees against a plain array across randomized replacements and queries.

23 **SDE-2 Follow-up:** Differential-test an AVL set against `TreeSet` after every insertion while independently verifying inorder order, stored height, and balance factors.

SDE-2 CORE CHAPTER

Chapter 8 - Trees, BSTs, and Tries Practice Lab Solutions

1 Apply the definitions consistently and state whether height counts edges or nodes; a leaf has height zero under edge height and one under node height.

2 Preorder processes before children, inorder between children, postorder after children, and BFS by depth. Trace actual node references rather than memorizing labels.

3 The BST global ordering invariant places all left-subtree keys before the node and all right-subtree keys after it.

4 With node-count height, null must return 0 and a leaf returns 1. Returning 1 makes a leaf height 2.

5 Carry ancestor bounds or verify strict inorder increase. Immediate children do not represent all descendants.

6 Preorder uses a stack and pushes right before left. Postorder can use two stacks, a `(node, expanded)` frame, or reverse a root-right-left sequence.

7 Return subtree height, but return a sentinel such as -1 when a child is unbalanced. Each node is processed once.

8 A postorder call returns height while updating a maximum of left height plus right height. State whether that sum is edges or adjust for node count.

9 Preorder plus explicit null markers uniquely preserves shape. Parsing consumes tokens in the same recursive contract; validate malformed/trailing input.

10 Iterative inorder counts visited nodes and stops at k. O(h + k) time and O(h) auxiliary space.

11 Recursively clear terminal at the word end, then remove a child only if it is nonterminal and has no children. Preserve nodes shared by longer or sibling words.

12 BFS takes the last node per level; right-first DFS records the first node seen at each depth.

13 Recursive depth becomes O(n) and can overflow the Java stack. Use an explicit deque or constrain input depth.

14 A hand-built BST gives ordering but not prefix structure; `TreeMap` is balanced and can support range queries; a trie follows prefixes directly but may use much more memory.

15 Options include parent pointers plus depth alignment, binary lifting, Euler tour plus RMQ, or offline algorithms. Choose from update frequency, memory, and query latency.

16 A child returns only its best nonnegative downward contribution because a parent can extend one branch. At the current node, update a global best with node value plus both nonnegative child contributions. Initialize below every node value so an all-negative tree selects its least-negative node. Time is O(n), with O(h) call depth.

17 Search from the root while recording the current node whenever moving left; it is the best greater ancestor seen so far. If the target has a right subtree, return that subtree's leftmost node. Return empty when no successor exists and reject or explicitly return empty for a missing target according to the API contract.

Range-query and balancing solutions

18 `12` is binary `1100`; `12 & -12` is `0100`, or 4. Fenwick cell 12 summarizes the four one-based positions `[9,12]`. In general it covers `[i-lowbit(i)+1, i]`.

19 Allocate `long[length+1]`. For public point index `p`, update internal indexes starting at `p+1` and repeatedly add `i & -i`. For a prefix endpoint `r`, start at internal `r` and repeatedly subtract the low bit. Range `[l,r)` is `prefix(r)-prefix(l)`. Validate `p` in `[0,n)` and endpoints in `[0,n]`; `[i,i)` returns zero. The complete code is `FenwickTree` in the companion.

20 Choose `leafBase` as the smallest power of two at least `max(1,n)`. Copy values to `tree[leafBase+i]`, build parents downward, and recompute ancestors after a point replacement. For `[l,r)`, translate both endpoints to leaves, consume an odd left node and the node before an odd right endpoint, then shift to parents. Each update/query touches `O(log n)` nodes; build is `O(n)`.

21 LL uses `rotateRight`, RR uses `rotateLeft`, LR rotates the left child left then the node right, and RL mirrors it. In a right rotation, the promoted left child's right subtree transfers to the demoted node's left because its keys lie between them. Update the demoted node's height before the promoted node. `AvlTree.rotationTrace()` makes each chosen case observable.

22 Keep a `long[]` oracle. On replacement, compute `delta = replacement - old`, apply Fenwick `add(index,delta)`, segment `set(index,replacement)`, and update the oracle. On a random half-open query, scan the oracle and compare both tree results. Generate empty ranges, negatives, endpoints zero/`n`, and repeated updates with a fixed seed.

23 Insert the same random value into the AVL and `TreeSet`. After every operation, compare inorder list and size, then recursively recompute child heights. Reject if a stored height differs, `abs(leftHeight-rightHeight)>1`, or inorder is not strictly increasing. Include duplicates to verify the selected set policy. The companion performs this check after each of one thousand insertions.

SDE-2 CORE CHAPTER

Chapter 9 - Executable Tree and Range-Tree Checks

This dependency-free Java 21 companion validates traversal and BST contracts, Fenwick and iterative segment trees, all AVL rotation families, height/balance invariants, and differential cases. A successful run prints PASS 18 tree checks.

JAVA (continued 1/12)

```java
import java.util.ArrayDeque;
import java.util.ArrayList;
import java.util.Arrays;
import java.util.Deque;
import java.util.List;
import java.util.Random;
import java.util.TreeSet;

public final class TreeInterviewChecks {
    static final class TreeNode {
        final int value;
        TreeNode left;
        TreeNode right;

        TreeNode(int value) {
            this.value = value;
        }
    }

    private TreeInterviewChecks() {}

    /** One-based internal storage; callers use zero-based indexes. */
    static final class FenwickTree {
        private final long[] tree;

        FenwickTree(int length) {
            if (length < 0) {
                throw new IllegalArgumentException("length cannot be negative");
            }
            tree = new long[length + 1];
        }

        FenwickTree(int[] values) {
            this(values.length);
            for (int index = 0; index < values.length; index++) {
```

JAVA (continued 2/12)

```
                add(index, values[index]);
            }
        }

        int length() {
            return tree.length - 1;
        }

        void add(int index, long delta) {
            requireIndex(index, length());
            for (int internal = index + 1; internal < tree.length;
                    internal += internal & -internal) {
                tree[internal] += delta;
            }
        }

        long prefixSum(int rightExclusive) {
            if (rightExclusive < 0 || rightExclusive > length()) {
                throw new IndexOutOfBoundsException("right endpoint is outside [0,n]");
            }
            long sum = 0;
            for (int internal = rightExclusive; internal > 0;
                    internal -= internal & -internal) {
                sum += tree[internal];
            }
            return sum;
        }

        long rangeSum(int left, int rightExclusive) {
            requireRange(left, rightExclusive, length());
            return prefixSum(rightExclusive) - prefixSum(left);
        }
    }
```

JAVA (continued 3/12)

```java
    /** Iterative point-update/range-sum segment tree with half-open queries. */
    static final class SegmentTree {
        private final int length;
        private final int leafBase;
        private final long[] tree;

        SegmentTree(int[] values) {
            length = values.length;
            int base = 1;
            while (base < Math.max(1, length)) {
                base <<= 1;
            }
            leafBase = base;
            tree = new long[leafBase << 1];
            for (int index = 0; index < length; index++) {
                tree[leafBase + index] = values[index];
            }
            for (int node = leafBase - 1; node > 0; node--) {
                tree[node] = tree[node << 1] + tree[node << 1 | 1];
            }
        }

        int length() {
            return length;
        }

        void set(int index, long value) {
            requireIndex(index, length);
            int node = leafBase + index;
            tree[node] = value;
            for (node >>= 1; node > 0; node >>= 1) {
                tree[node] = tree[node << 1] + tree[node << 1 | 1];
            }
        }
```

JAVA (continued 4/12)

```
        long rangeSum(int left, int rightExclusive) {
            requireRange(left, rightExclusive, length);
            long leftSum = 0;
            long rightSum = 0;
            int first = left + leafBase;
            int afterLast = rightExclusive + leafBase;
            while (first < afterLast) {
                if ((first & 1) == 1) {
                    leftSum += tree[first++];
                }
                if ((afterLast & 1) == 1) {
                    rightSum = tree[--afterLast] + rightSum;
                }
                first >>= 1;
                afterLast >>= 1;
            }
            return leftSum + rightSum;
        }

        long[] levelOrderStorageForTeaching() {
            return tree.clone();
        }
    }

    /** Minimal set-style AVL tree that records which balancing rotation ran. */
    static final class AvlTree {
        private AvlNode root;
        private int size;
        private final List<String> rotationTrace = new ArrayList<>();

        int size() {
            return size;
        }
```

JAVA (continued 5/12)

```
        Integer rootKey() {
            return root == null ? null : root.key;
        }

        List<String> rotationTrace() {
            return List.copyOf(rotationTrace);
        }

        void add(int key) {
            int previousSize = size;
            root = insert(root, key);
            if (size == previousSize) {
                rotationTrace.add("duplicate " + key + " ignored");
            }
        }

        List<Integer> inorder() {
            List<Integer> values = new ArrayList<>();
            collectInorder(root, values);
            return values;
        }

        boolean isHeightBalanced() {
            return verifiedHeight(root) >= 0;
        }

        private AvlNode insert(AvlNode node, int key) {
            if (node == null) {
                size++;
                return new AvlNode(key);
            }
            if (key < node.key) {
                node.left = insert(node.left, key);
```

JAVA (continued 6/12)

```
            } else if (key > node.key) {
                node.right = insert(node.right, key);
            } else {
                return node;
            }
            updateHeight(node);
            int balance = balance(node);
            if (balance > 1) {
                if (key > node.left.key) {
                    rotationTrace.add("LR: rotateLeft(" + node.left.key + ")");
                    node.left = rotateLeft(node.left);
                }
                rotationTrace.add("rotateRight(" + node.key + ")");
                return rotateRight(node);
            }
            if (balance < -1) {
                if (key < node.right.key) {
                    rotationTrace.add("RL: rotateRight(" + node.right.key + ")");
                    node.right = rotateRight(node.right);
                }
                rotationTrace.add("rotateLeft(" + node.key + ")");
                return rotateLeft(node);
            }
            return node;
        }

        private static AvlNode rotateRight(AvlNode top) {
            AvlNode promoted = top.left;
            AvlNode transferred = promoted.right;
            promoted.right = top;
            top.left = transferred;
            updateHeight(top);
            updateHeight(promoted);
            return promoted;
```

JAVA (continued 7/12)

```
        }

        private static AvlNode rotateLeft(AvlNode top) {
            AvlNode promoted = top.right;
            AvlNode transferred = promoted.left;
            promoted.left = top;
            top.right = transferred;
            updateHeight(top);
            updateHeight(promoted);
            return promoted;
        }

        private static int height(AvlNode node) {
            return node == null ? 0 : node.height;
        }

        private static int balance(AvlNode node) {
            return height(node.left) - height(node.right);
        }

        private static void updateHeight(AvlNode node) {
            node.height = 1 + Math.max(height(node.left), height(node.right));
        }

        private static void collectInorder(AvlNode node, List<Integer> output) {
            if (node == null) {
                return;
            }
            collectInorder(node.left, output);
            output.add(node.key);
            collectInorder(node.right, output);
        }

        private static int verifiedHeight(AvlNode node) {
```

JAVA (continued 8/12)

```
            if (node == null) {
                return 0;
            }
            int leftHeight = verifiedHeight(node.left);
            int rightHeight = verifiedHeight(node.right);
            if (leftHeight < 0 || rightHeight < 0
                    || Math.abs(leftHeight - rightHeight) > 1
                    || node.height != 1 + Math.max(leftHeight, rightHeight)) {
                return -1;
            }
            return 1 + Math.max(leftHeight, rightHeight);
        }
    }

    static final class AvlNode {
        private final int key;
        private int height = 1;
        private AvlNode left;
        private AvlNode right;

        AvlNode(int key) {
            this.key = key;
        }
    }

    static boolean isValidBst(TreeNode root) {
        return valid(root, Long.MIN_VALUE, Long.MAX_VALUE);
    }

    private static boolean valid(TreeNode node, long lower, long upper) {
        if (node == null) {
            return true;
        }
        return node.value > lower && node.value < upper
```

JAVA (continued 9/12)

```
                && valid(node.left, lower, node.value)
                && valid(node.right, node.value, upper);
    }

    static List<Integer> inorder(TreeNode root) {
        List<Integer> output = new ArrayList<>();
        Deque<TreeNode> stack = new ArrayDeque<>();
        TreeNode current = root;
        while (current != null || !stack.isEmpty()) {
            while (current != null) {
                stack.push(current);
                current = current.left;
            }
            current = stack.pop();
            output.add(current.value);
            current = current.right;
        }
        return output;
    }

    private static void requireIndex(int index, int length) {
        if (index < 0 || index >= length) {
            throw new IndexOutOfBoundsException("index " + index + " outside [0," + length + ")");
        }
    }

    private static void requireRange(int left, int rightExclusive, int length) {
        if (left < 0 || left > rightExclusive || rightExclusive > length) {
            throw new IndexOutOfBoundsException("invalid half-open range");
        }
    }

    private static boolean rangeTreesMatchArrayOnRandomOperations() {
        Random random = new Random(13L);
```

JAVA (continued 10/12)

```
        int[] values = new int[24];
        for (int index = 0; index < values.length; index++) {
            values[index] = random.nextInt(101) - 50;
        }
        FenwickTree fenwick = new FenwickTree(values);
        SegmentTree segment = new SegmentTree(values);
        long[] expected = Arrays.stream(values).asLongStream().toArray();
        for (int operation = 0; operation < 2_000; operation++) {
            if (random.nextBoolean()) {
                int index = random.nextInt(values.length);
                int replacement = random.nextInt(201) - 100;
                long delta = replacement - expected[index];
                expected[index] = replacement;
                fenwick.add(index, delta);
                segment.set(index, replacement);
            } else {
                int first = random.nextInt(values.length + 1);
                int second = random.nextInt(values.length + 1);
                int left = Math.min(first, second);
                int right = Math.max(first, second);
                long sum = 0;
                for (int index = left; index < right; index++) {
                    sum += expected[index];
                }
                if (fenwick.rangeSum(left, right) != sum
                        || segment.rangeSum(left, right) != sum) {
                    return false;
                }
            }
        }
        return true;
    }

    private static boolean avlMatchesTreeSetOnRandomInputs() {
```

JAVA (continued 11/12)

```
        Random random = new Random(17L);
        AvlTree avl = new AvlTree();
        TreeSet<Integer> expected = new TreeSet<>();
        for (int operation = 0; operation < 1_000; operation++) {
            int value = random.nextInt(401) - 200;
            avl.add(value);
            expected.add(value);
            if (!avl.isHeightBalanced()
                    || !avl.inorder().equals(new ArrayList<>(expected))) {
                return false;
            }
        }
        return avl.size() == expected.size();
    }

    private static void check(boolean condition, String message) {
        if (!condition) {
            throw new AssertionError(message);
        }
    }

    public static void main(String[] args) {
        TreeNode root = new TreeNode(8);
        root.left = new TreeNode(3);
        root.right = new TreeNode(10);
        root.left.left = new TreeNode(1);
        root.left.right = new TreeNode(6);
        check(isValidBst(root), "valid BST");
        check(inorder(root).equals(List.of(1, 3, 6, 8, 10)), "inorder");
        root.left.right.right = new TreeNode(12);
        check(!isValidBst(root), "ancestor violation");

        FenwickTree fenwick = new FenwickTree(new int[] {2, -1, 4, 3, 5});
        check(fenwick.prefixSum(0) == 0, "empty Fenwick prefix");
```

JAVA (continued 12/12)

```
        check(fenwick.rangeSum(1, 4) == 6, "Fenwick half-open range");
        fenwick.add(2, -4);
        check(fenwick.rangeSum(0, 5) == 9, "Fenwick point delta");

        SegmentTree segment = new SegmentTree(new int[] {2, -1, 4, 3, 5});
        check(segment.rangeSum(1, 4) == 6, "segment range");
        segment.set(2, 0);
        check(segment.rangeSum(0, 5) == 9, "segment point replacement");
        check(segment.rangeSum(3, 3) == 0, "empty segment range");
        check(segment.levelOrderStorageForTeaching().length == 16, "power-of-two storage");

        AvlTree leftLeft = new AvlTree();
        leftLeft.add(30);
        leftLeft.add(20);
        leftLeft.add(10);
        check(leftLeft.rootKey() == 20, "LL rotation root");
        check(leftLeft.rotationTrace().equals(List.of("rotateRight(30)")), "LL trace");

        AvlTree leftRight = new AvlTree();
        leftRight.add(30);
        leftRight.add(10);
        leftRight.add(20);
        check(leftRight.rootKey() == 20, "LR rotation root");
        check(leftRight.rotationTrace().equals(
                List.of("LR: rotateLeft(10)", "rotateRight(30)")), "LR trace");
        leftRight.add(20);
        check(leftRight.size() == 3, "duplicate ignored");
        check(leftRight.isHeightBalanced(), "AVL invariant");
        check(rangeTreesMatchArrayOnRandomOperations(), "range-tree differential test");
        check(avlMatchesTreeSetOnRandomInputs(), "AVL differential test");

        System.out.println("PASS 18 tree checks");
    }
}
```

Part II - Practice and Handoff

TRY IT | Practice Ladder and Next Steps

TRY IT | Practice ladder

- Foundation: traversals, height, and search
- Interview Core: validate BST, LCA, and path problems
- SDE-2 Follow-up: iterative traversal and state ownership
- Challenge: serialization or prefix-search design

For every coding problem, state the input contract, select the numeric and collection types deliberately, write the invariant before the loop or recursion, test the smallest and largest valid inputs, and close with time and auxiliary-space complexity.

Navigation

Previous book: DSA 12 - Binary Search: Bounds, Answers, and Invariants

Series index

Next book: DSA 14 - Heaps, Priority Queues, Selection, and Top-K

TRY IT | Completion check

- [ ] I can recognize the core patterns without being told the data structure or algorithm.
- [ ] I can implement the representative Java templates from memory.
- [ ] I can explain why each implementation is correct.
- [ ] I can test boundaries, invalid inputs, overflow, and adversarial shapes.
- [ ] I can answer at least one SDE-2 follow-up involving trade-offs or production constraints.

Series Roadmap

Choose Java, DSA, Frameworks, or System Design first, then follow the books inside that segment in order. The same series codes and positions appear on the website, PDF covers, and canonical download folders.

SEGMENT	BOOK	FOCUSED LEARNING PATH
Java	**JAVA 01**	Java Foundations for Problem Solving
Java	**JAVA 02**	Git and GitHub for Java Engineers
Java	**JAVA 03**	Maven and Gradle for Java Engineers
Java	**JAVA 04**	Language, OOP, and Modern Java
Java	**JAVA 05**	Collections, Streams, and I/O
Java	**JAVA 06**	JVM and Java Execution
Java	**JAVA 07**	Java Concurrency and the Memory Model
Java	**JAVA 08**	Performance, Diagnostics, and GC Incidents
Java	**JAVA 09**	Java Question Bank, Study Plan, and Reference
DSA	**DSA 01**	Time and Space Complexity for Java Interviews
DSA	**DSA 02**	Number Systems and Math Foundations for DSA Interviews
DSA	**DSA 03**	Number Systems Interview Patterns and Rapid Revision
DSA	**DSA 04**	Bit Manipulation in Java
DSA	**DSA 05**	Loop Mastery, Patterns, and Index Calculations
DSA	**DSA 06**	Arrays and Array Problem-Solving Patterns
DSA	**DSA 07**	Strings and String Problem-Solving Patterns
DSA	**DSA 08**	Hashing: Maps, Sets, Frequency, and Prefix State
DSA	**DSA 09**	Recursion and Backtracking in Java
DSA	**DSA 10**	Linked Lists: Pointer Reasoning and Mutation
DSA	**DSA 11**	Stacks, Queues, Deques, and Monotonic Patterns
DSA	**DSA 12**	Binary Search: Bounds, Answers, and Invariants
DSA	**DSA 13**	Trees, Binary Search Trees, and Tries
DSA	**DSA 14**	Heaps, Priority Queues, Selection, and Top-K
DSA	**DSA 15**	Graphs: Traversal, Ordering, Paths, and Union-Find
DSA	**DSA 16**	Greedy Algorithms: Recognition, Proofs, and Scheduling
DSA	**DSA 17**	Dynamic Programming: State, Transitions, and Optimization
Frameworks	**FW 01**	MySQL for Java Backend Interviews
Frameworks	**FW 02**	Hibernate and JPA for Java Backend Interviews
Frameworks	**FW 03**	Spring Framework for Java Engineers
Frameworks	**FW 04**	Spring Boot for Java Backend Engineers
Frameworks	**FW 05**	Spring Boot and REST APIs

SEGMENT	BOOK	FOCUSED LEARNING PATH
Frameworks	**FW 06**	Spring Data for Java Backend Engineers
Frameworks	**FW 07**	MongoDB for Java Backend Interviews
Frameworks	**FW 08**	Redis for Java Backend Interviews
Frameworks	**FW 09**	Persistence, SQL, JPA, and Caching
Frameworks	**FW 10**	Apache Kafka and Spring Kafka
Frameworks	**FW 11**	Spring Ecosystem Extensions
Frameworks	**FW 12**	Spring AI for Java Engineers
System Design	**SD 01**	Design, Backend, Testing, and Security
System Design	**SD 02**	Distributed Systems and System Design
System Design	**SD 03**	Generative AI System Design

All 40 publication editions are available now. Choose Java, DSA, Frameworks, or System Design first, then follow the books inside that shelf in order. Each deep-dive book remains independently printable and available on the web.

About the Author

Vinay Reddy Kalluri

Vinay Reddy Kalluri is a senior Java backend engineer and the author and editor of the Java SDE-2 Interview Preparation Series. His work spans high-throughput microservices, event-driven architecture, resilient APIs, large-scale data processing, cloud delivery, and production performance across healthcare and enterprise systems.

Editorial approach

Vinay sets the learning sequence and publication standard and performs the final review for Java accuracy, evidence, attribution, and release readiness. Individual contributors retain visible credit for accepted original work through AUTHORS.md and the public Git history.

What distinguishes his approach is the combination of hands-on system design and operational ownership. He treats timeouts, retries, idempotency, dead-letter recovery, data consistency, SQL behavior, caching, and observability as part of the design rather than afterthoughts. He has also mentored engineers and led modernization, migration, and reliability work across distributed services.

Selected engineering and academic highlights

- More than eight years building Java and Spring Boot services for high-throughput healthcare and enterprise platforms.
- Designed Kafka-based event systems that have processed more than one million events per hour, with explicit idempotency, retry, recovery, and observability controls.
- M.S. in Computer Science from the University of Missouri-Kansas City (3.9/4.0) and M.S. in Software Engineering from VIT University (9.3/10), where he was a Gold Medalist.
- Independent builder of Work Visa Insights, an immigration-data intelligence platform, and Play Together, a published Android application.

Why this series exists

Vinay created this series to turn fragmented interview preparation into a deliberate progression: learn the fundamentals, run the examples, predict behavior, debug failures, explain trade-offs, and only then move into advanced engineering. The books reflect the same principle he applies to production systems: clarity before cleverness, explicit contracts, measurable behavior, and reliable foundations.

Connect: LinkedIn | GitHub

Copyright and Publishing Notes

Examples target Java 21 unless a section states otherwise. Validate release-specific APIs, JVM behavior, security, performance, licensing, and operational requirements before production use.

Series position: Data Structures and Algorithms - Book 13 of 17 - Study Step DSA-13. The local table of contents and PDF bookmarks navigate within this file; the roadmap and printed filenames navigate across the complete series.

Source provenance: curated from the independent Java SDE-2 master guide plus focused original material. Original master chapter numbers are labeled explicitly when retained in cross-references.